

실데이터 사전 시뮬레이션

WaterPulse를 실제 공공 데이터로 처음부터 끝까지 한 번 돌려 봤습니다.

694만 줄

공공 API로 받은 실제 데이터

38분

수집 (호출 16,146번)

3,344번

분석에 쓴 큰비 사건

진행해도

됩니다

단, 예측 기대치와 추세
분석은 조정

K-water 댐·보 59곳의 시간 자료 6년치와 일 자료 20년치, 상류 관측소 72곳의 자료를 받아
수집 → 적재 → 정리·저장 → 분석 → 웹까지 과제와 같은 순서로 해 봤습니다.

이 문서에서 볼 수 있는 것

무엇을 어떻게 받았고, 어떻게 정리·저장했고, 무엇을
분석했는지와 그 결과. 그리고 실제 과제에서 그대로 따라 하는
법.

결과 웹페이지

<https://keemlogan.github.io/waterpulse/>
댐을 누르면 반응 시간·큰비 그래프·예측 성능이 나옵니다.

과제를 진행해도 될까요? — 네

실제 데이터로 확인한 7가지 질문의 답입니다.

질문	실제로 해 보니	판정
데이터를 정말 받을 수 있나?	공공 API 16,146번 호출로 6,944,601줄. 최종 실패 0건(오류는 재시도로 해결).	예
하둡 방식으로 정리·저장이 되나?	함정 5가지를 실제로 만나고 처리. 시간 자료 429MB → 47MB(Parquet).	예
반응 시간(비교 분석)이 의미 있게 나오나?	큰비 3,344번, 댐 53곳. 빗물이 모이는 땅(유역)이 넓을수록 늦게 몰려옴. 넓이 순서와 반응 시간 순서가 거의 같음(순위 상관 $\rho=0.89$, 1이면 완전히 같은 순서, 댐 35곳).	예
예측이 "지금 그대로"보다 나은가?	"지금 양 그대로" 찍기가 100만큼 틀릴 때 모델은 6시간 뒤를 90만큼 틀림(+10%). 홍수 때(그 댐 유입 상위 5% 시간)는 +17%. 1차 시도는 +7%였음.	조건부
상류 관측소가 도움이 되나?	소양강·대청·안동 3곳 모두 6시간 예측이 나아짐. 덜 틀리는 정도가 가운데 값 기준 +2.9%포인트 더 늘어남(예: 14% → 17%). 작지만 일관됨.	예
20년 추세가 보이나?	해마다 홍수 크기(그해 최대 유입 ÷ 보통 해)에 커지거나 작아지는 흐름 없음($\tau=-0.04$: 0이면 흐름 없음, $p=0.82$: 우연으로도 흔히 나오는 수준). 2020년이 가장 컸음.	아니오
이상한 순간을 찾을 수 있나?	예측과 실재가 평소보다 훨씬 크게 벌어진 때를 2024-2025년에 1,157번 표시. 자료 오류인지 진짜 변화인지 나눠야 함.	예

⚠ 기대와 달랐던 것 3가지 → 이렇게 바꾸면 됩니다

예측이 생각보다 약함

매시간 유입량은 계산값이라 들쭉날쭉합니다. **3시간 평균**을 맞게 바꾸니 나아졌습니다. 발표에선 "홍수 때(유입 상위 5% 시간) 얼마나 더 정확한가"를 앞세우기.

20년 추세가 안 보임

홍수 크기는 해마다 크게 출렁입니다. 추세 대신 "**특별했던 해(2020·2006·2023)**" 비교와 계절 변화로 방향을 바꾸기.

이상한 유입량이 많음

0 이하가 댐별 가운데 값 13%, 불가능한 값도 17개(광동댐 4,223만 m^3/s). 거르는 규칙을 정하고 발표의 "**예외 처리**"로 쓰기.

한 줄 결론: 데이터·하둡 처리·비교 분석은 기대대로 됩니다. 예측과 추세는 "기대치 조정"이 필요하지만, 그 과정 자체가 최종발표 채점 항목인 "문제 분석과 디버깅"의 좋은 재료입니다.

이 문서에 나오는 말

숫자와 기호가 처음 나올 때 막히지 않도록, 뒤 쪽에서 쓰는 말을 먼저 풀어 둡니다.

반응 시간	비가 가장 세게 내린 시각부터 댐에 물이 가장 많이 들어온 시각까지 걸린 시간.
가운데 값	값들을 크기 순으로 줄 세웠을 때 한가운데 값. 유난히 튀는 값 하나에 끌려가지 않아 평균 대신 씀.
유역 넓이	비가 내렸을 때 그 댐으로 물이 흘러드는 땅의 넓이(km ²).
“지금 그대로” 찍기 (기준선)	앞으로도 지금 들어오는 양과 같다고 찍는 가장 단순한 예측. 내일 날씨를 “오늘과 같다”고 찍는 것과 같음. 모델은 이걸 이겨야 의미가 있음.
오차가 X% 줄었다	찍기가 평균 100만큼 틀릴 때 모델이 100-X만큼 틀렸다는 뜻. 예: 10% → 90만큼 틀림. 0보다 작으면 찍기보다 못함.
%포인트(%p)	퍼센트끼리의 차이. 14%에서 17%가 되면 3%포인트 늘었다고 씀.
홍수 때 (상위 5%)	맞혀야 할 유입량이 그 댐의 평소 기록 중 가장 많은 5% 안에 드는 시간. 이전 판의 “큰물”.
홍수 크기	그해 가장 많이 들어온 양 ÷ 그 댐의 보통 해 값. 1이면 보통 해, 2면 보통의 2배. 이전 판의 “큰물 지수”.
순위 상관 ρ (스피어만)	두 가지 기준으로 각각 줄 세웠을 때 순서가 얼마나 같은지. 1이면 똑같은 순서, 0이면 관계 없음, -1이면 정반대.
τ (켄달 타우)	해가 갈수록 값이 커지는지(+) 작아지는지(-). 0에 가까우면 흐름이 없음.
p값	실제로는 아무 관계가 없는데도 우연히 이런 결과가 나올 확률. 0.05보다 작으면 “우연은 아닌 듯”, 크면 “우연으로도 나올 수 있음”.
교차상관으로 검산	비 그래프를 몇 시간 뒤로 밀었을 때 유입 그래프와 가장 닮는지 찾는 다른 방법. 반응 시간을 두 방법으로 재서 맞춰 봄.
보 · 조정지 · 하굿둑	보 = 강을 가로막은 낮은 둑, 조정지 = 큰 댐 바로 아래에서 방류를 받아 고르게 흘려보내는 작은 저수지, 하굿둑 = 강 끝에서 바닷물을 막는 둑.
이상 탐지	예측과 실제가 평소보다 훨씬 크게 벌어진 시간(차이가 2023년 기준 가장 큰 0.5% 안)을 “평소와 다름”으로 표시하는 것. 사고라는 뜻이 아님.

우리가 실제로 한 일

과제의 흐름(수집 → 관리 → 분석 → 웹)을 그대로 따랐습니다. 숫자는 모두 실제 실행 기록입니다.



실제로 만들어진 폴더 (HDFS 대신 노트북 폴더)

```

WaterPulse/sim/
├─ collect.py · flume_sim.py · etl_spark.py · analyze.py  ← 실행한 코드 4개
├─ spool/          수집기가 떨어진 파일 (보낸 뒤엔 .COMPLETED 표시만 남음)
├─ lake/etl/waterpulse/raw/          ← 원본 선반 (수업 폴더 규칙 /etl)
│   ├── dam_hourly/damcode=1012110/year=2023/events.jsonl
│   ├── dam_daily/ · rain_hourly/ · level_hourly/
├─ lake/data/waterpulse/clean/      ← 정리 선반 (Parquet, /data)
│   └─ dam_hourly/year=2023/month=7/part-....parquet
├─ lake/data/waterpulse/mart/      ← 결과 선반 (학습표)
├─ logs/calls.jsonl             호출 한 번 = 한 줄 기록 (16,230줄)
└─ out/                         분석 결과 JSON → 웹·이 문서가 읽음
    
```

🕒 걸린 시간

수집 38분 · 적재 24초 ·
정리 1분 미만 · 분석
10분

📁 돌린 곳

MacBook Air M2 ·
Spark 4.2 (로컬 모드) ·
Java 21 · Python 3.12

🔑 키 관리

인증키는
과제/.env에서만 읽고,
기록·결과 어디에도
남기지 않음

① 무엇을, 어떻게 받았나

Python 수집기(collect.py)가 공공데이터포털 K-water API를 불러 받은 그대로 파일로 저장했습니다.

자료	범위	호출	받은 줄	걸린 시간
댐 일 자료 수문 운영 정보 · 일	59곳 × 2006-2025	1,183	395,547	8분
댐 시간 자료 수문 운영 정보 · 시간	59곳 × 2020-2025	7,789	3,103,845	6분
상류 비 우량수위 관측소 · 시강우	39곳 × 2020-2025	4,350	2,051,478	함께 24분
상류 강 우량수위 관측소 · 시수위	33곳 × 2020-2025	2,900	1,393,731	
기준표 댐코드 · 수문 제원 · 관측소 목록	59댐 · 제원 60 · 3댐 목록	8	-	수 초

✓ 수집기 규칙 (그대로 쓰면 됩니다)

- 한 번에 댐 400줄, 관측소 500줄씩 쪽을 넘겨 받기
- 댐 API는 6갈래 동시, 관측소 API는 3갈래 + 0.3초 간격
- 빈 답·한도 초과면 쉬었다 최대 7번 다시
- **댐×연 단위 파일 하나로 저장**(작은 파일 문제 피하기)
- 다 쓴 파일만 보이게 저장(.tmp → .json)
- 이미 받은 파일은 건너뛰어 끊겨도 이어 받기

📦 실제로 만난 문제

- **초당 한도 초과(429, 코드 23)**: 관측소 API에 6갈래로 부르니 막힘 → 3갈래 + 간격으로 해결 (59번 발생)
- **999줄 요청이 어떤 쪽에선 빈 답**: 1쪽은 되는데 6쪽은 0바이트 → 500줄로 낮춰 해결 (21번 발생)
- 네트워크 오류 4번 → 재시도로 해결
- 응답 시간 가운데 값: 댐 0.19초, 관측소 0.2초

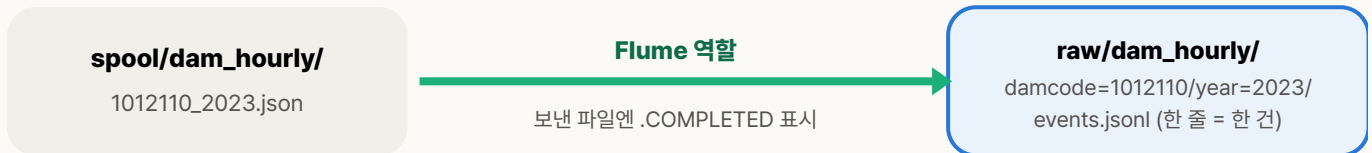
📄 받은 그대로의 한 줄 (소양강댐, 2023년)

```
{"inflowqy": 495.698, "lowlevel": 181.27, "obsrdt": "07-15 21시 ", "rf": "0.100", "rsvwtqy": "1,779.649", "rsvwtrt": 61.4, "totdcwtrqy": 199.948}
```

→ 시각에 연도가 없고("07-14 ...시"), 저수량은 쉼표가 든 글자("1,...")입니다. 이런 값을 고치는 일은 수집기가 아니라 Spark가 합니다.

② 창고에 넣기 — Flume이 하는 일

flume_sim.py가 Flume의 spooldir처럼 수집 폴더를 비우며 원본 선반으로 옮겼습니다.



1차 적재

1,534개 파일 ·
3,499,392줄 ·
486MB · 15초
댐 일·시간 자료(첫
번째 적재)

2차 적재

432개 파일 ·
3,445,209줄 ·
250MB · 9초

왜 이렇게 옮기나

- 한 줄 = 측정값 한 건이면
Hive·Spark가 바로 읽음
- 댐·연도는 **폴더 이름**으로 → 나중에
필요한 칸만 읽기
- 수집과 적재가 따로 돌아도 서로 안
꼬임(.COMPLETED)

실제 VM에서는 (9장에서 배울 Flume 설정의 모양)

```

agent.sources.s1.type = spo
oldir
agent.sources.s1.spoolDir =
/home/.../spool/dam_hourly
agent.sinks.k1.type = hdfs
agent.sinks.k1.hdfs.path =
/etl/waterpulse/raw/dam_hou
rly
# 배우기 전엔: hdfs dfs -put ...
/etl/waterpulse/raw/
  
```

③ 정리 — 데이터 합정을 실제로 만났고, 이렇게 처리했습니다

etl_spark.py가 원본 표(전부 STRING)를 읽어 고친 뒤 정리 선반에 저장했습니다.

“그냥 숫자로 바꿨다면” 사라졌을 값

40,443 131,738 38,958

방류량

침표가 든 숫자("1,946.781")는 숫자 열에서 오류 없이 NULL이 됩니다(Hive의 schema on read와 같은 동작). 침표를 빼고 바꾸니 0개.

새로 찾은 합성 ㉮ 불가능한 값: 광동댐 2020-11-07
10:00 유입량 **42,231,464m³/s**. 이 한 줄 때문에
그 댐 예측이 망가졌습니다. 규칙: 그 댐 평소 최댓값
(상위 0.1%)의 10배 초과 + 앞뒤 1시간보다 10배
넘게 틈 → 빈칸.

합정별 실제 건수

“24시” → 다음 날 0시	129,327건
시각에 연도 붙이기	시간 자료 전부
유입량 0 이하 (표시만)	401,906건 (12.9%)
불가능한 유입량 → 빈칸	17건
유입량 빈칸 (위 포함)	100건
중복 시각 제거	2건
시험용 관측소 빼기	“시험2” 1곳

원본 표 만들기 (Hive와 같은 SQL, 모든 열 STRING)

```
CREATE TABLE raw_dam_hourly (inflowqy STRING, lowlevel STRING, obsrdt STRING, rf STRING, rsvwtqy STRING, rsvwtrt STRING, totdcwtrqy STRING, damcode STRING, year INT) USING json PARTITIONED BY (damcode, year) LOCATION '.../lake/etl/waterpulse/raw/dam_hourly'
```

⚙️ Spark에서 고친 방법 (핵심 세 줄)

```
ts = make_timestamp(year, 월, 일, 0, 0, 0) + make_interval(0,0,0,0, 시) # 24시 → 다음
날 0시가 저절로
num = regexp_replace(trim(col), ",", "").cast("double") # 쉼표 빼
고 숫자로
dam_h.repartition("year","month").write.partitionBy("year","month").parquet("../clea
n/dam_hourly")
```

💡 알게 된 것: Spark 4는 기본이 "엄격 모드"라 심표 숫자를 만나면 오류를 내고 멈춥니다. 수업의 Hive처럼 조용히 NULL로 만들려면 `spark.sql.ansi.enabled=false` . 실습 VM의 Spark 버전에 따라 동작이 다를 수 있으니 확인이 필요합니다.

④ 저장 — 압축 형식과 칸 나누기의 효과

같은 데이터를 원본 JSON, Parquet, 연·월로 칸 나눈 Parquet으로 두고 비교했습니다.

자료	원본 JSON	Parquet	줄어든 정도
댐 시간 자료	354개 · 429MB	73개 · 47MB	9.2배
댐 일 자료	1,180개 · 57MB	20개 · 9MB	6.1배
상류 비	234개 · 149MB	6개 · 4MB	36.0배
상류 강	198개 · 101MB	6개 · 6MB	18.3배

파일 수도 줄었습니다: 일 자료는 댐×연 파일 1,180개가 연도별 20개로. HDFS가 싫어하는 “작은 파일 많음”을 정리 단계에서 해결한 것입니다.

🕒 같은 질문, 다른 저장 방식 (5번 실행한 가운데 값)

질문	원본 JSON	Parquet	Parquet + 연·월 칸
2023년 7월, 전체 댐 평균 유입량	140ms	66ms	35ms
6년 전체, 댐별 평균 유입량	892ms	73ms	115ms

실행 계획에 남은 증거: PartitionFilters: [isnotnull(year#4768), isnotnull(month#4769), (year#4768 = 2023), (month#4769 = 7)]

읽는 법

- 한 달만 묻는 질문은 칸 나누기로 해당 칸만 읽음
- 전체를 훑는 질문은 Parquet(열 단위)이 JSON보다 훨씬 빠름
- 다만 전체를 훑을 땐 칸 나눈 쪽이 파일이 많아 조금 느릴 수 있음 → 자주 묻는 질문에 맞춰 칸을 정하기
- 용량은 Parquet이 약 9분의 1

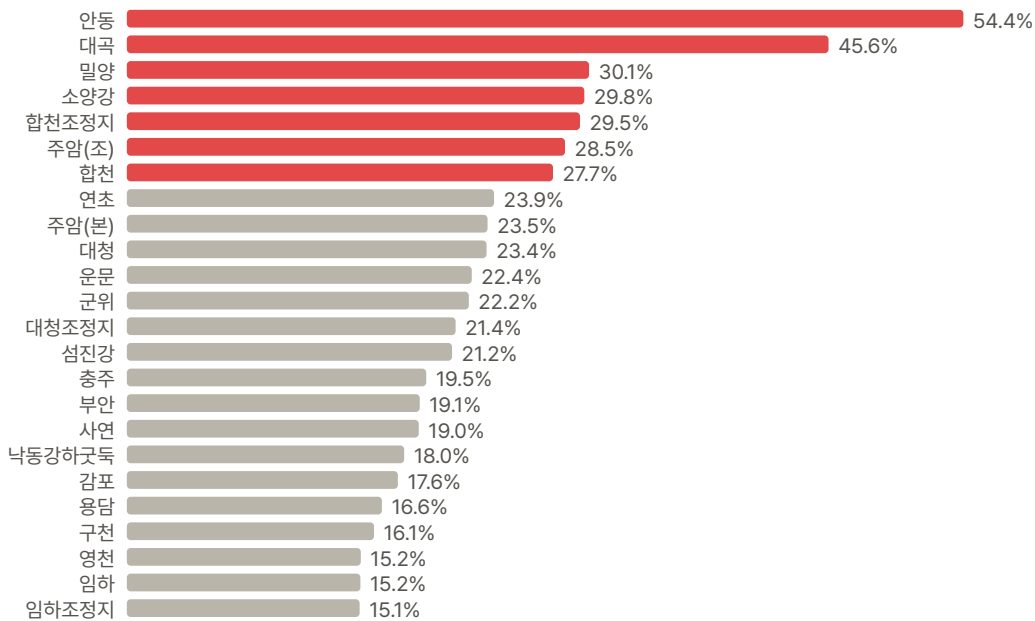
주의

노트북 한 대의 로컬 실행이라 시간 차이가 작게 나옵니다. 실습 VM의 HDFS·Impala에서 같은 비교를 다시 재서 발표에 쓰세요. 방법은 같습니다.

분석 전에: 데이터 품질표

시간 자료는 59곳 모두 빠진 시간 없이 100% 있었습니다. 문제는 “0 이하 유입량”입니다.

유입량이 0 이하인 시간의 비율 (많은 순 24곳)



왜 생기나

유입량은 잔 값이 아니라 **저수량 변화 + 방류량**으로 계산한 값입니다. 비가 없는 시기엔 증발·측정 오차로 0이나 음수가 자주 나옵니다. 안동·대곡처럼 절반이 넘는 곳도 있습니다.

이렇게 처리했습니다

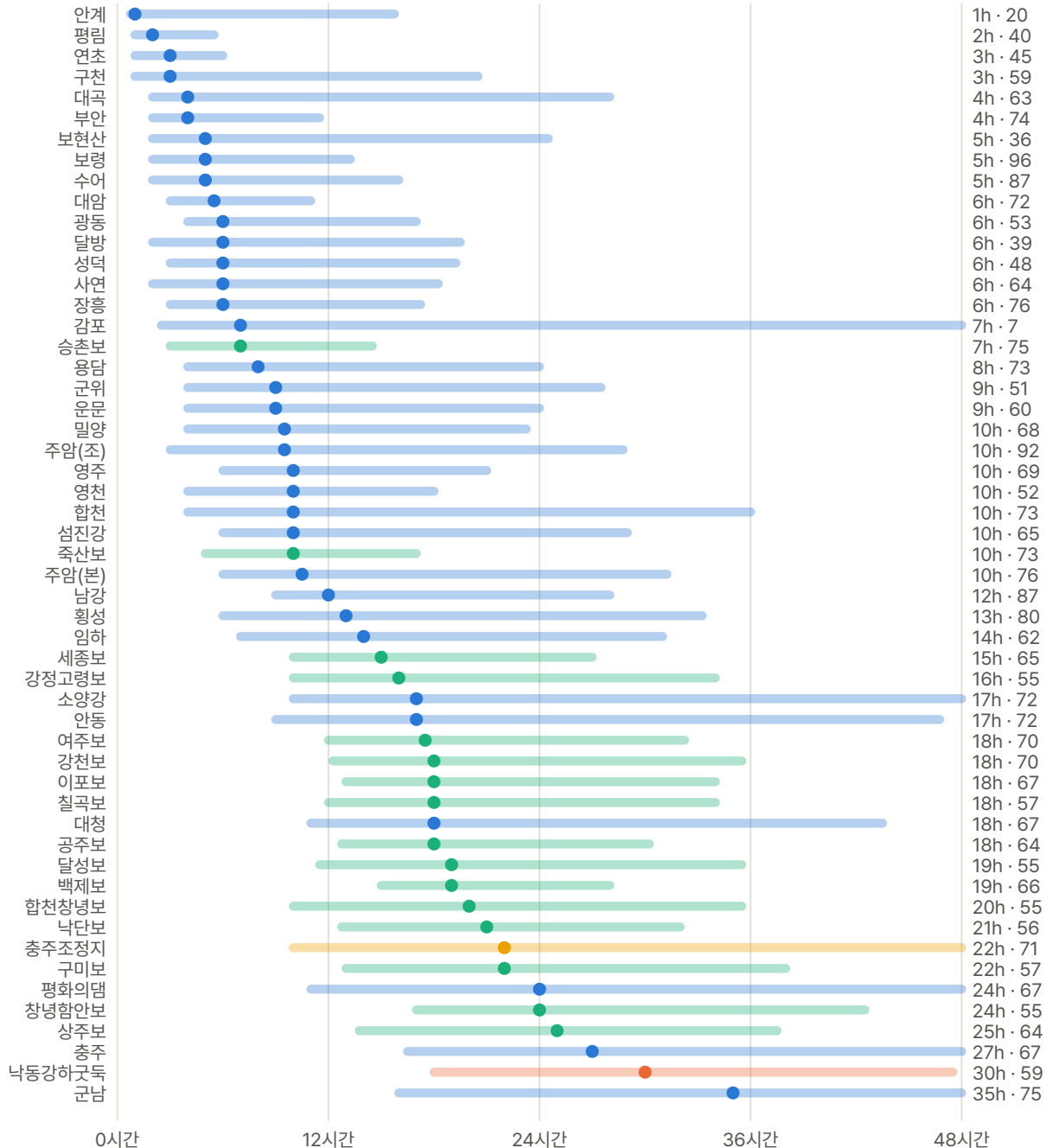
- 반응 시간: 0 이하는 빼고 3시간 평균으로 봄
- 예측: 지난 3시간 중 양수인 값의 평균 (s3)을 씀
- 웹·발표: 댐마다 비율을 그대로 공개
→ “예외를 알고 있다”는 증거

분석 ① 큰비 뒤 몇 시간 만에 물이 몰려오나

큰비 사건 3,344번을 찾아, 사건마다 “비 최고 → 유입 최고” 시간을 쟀습니다. 사건 5번 이상인 53곳의 가운데 값입니다. 반응 시간 = 비가 가장 세게 내린 시각 → 댐에 물이 가장 많이 들어온 시각.

■ 댐 ■ 보 ■ 조정지 ■ 하굿둑

· 한 줄 = 댐 하나 · 점 = 큰비마다 쟀 시간의 가운데 값 · 열은 막대 = 큰비 절반이 들어가는 범위(길면 비마다 들쭉날쭉) · 오른쪽 “17h · 72” = 17시간, 큰비 72번



사건을 자른 규칙

- 비가 6시간 넘게 났으면 다른 사건
- 사건 강우 합 30mm 이상만
- 유입이 평소(직전 24시간)의 1.5배·5m³/s 이상 늘어난 사건만
- 비 최고 뒤 72시간 안의 유입 최고를 찾음

다른 방법으로 검산

비 그래프를 몇 시간 뒤로 밀었을 때 유입 그래프와 가장 닮는지 찾는 다른 방법(교차상관, 6~9월)과 비교: **25/53**곳이 3시간 안으로 일치. 교차상관이 대체로 더 짧게 나와, 발표에선 사건 기반 값을 쓰고 차이를 한계로 적기.

예: 소양강댐에 가장 큰 물이 들어온 때

2024-09-19 17:00 ~ 2024-09-23 00:00 · 실제 값 (비 = K-water 댐 강우, 선 = 유입량)

비 (mm/시간)

유입량 (m³/s)

12

1,904

952

0

2024-09-19 17:00

2024-09-23 00:00

19시간

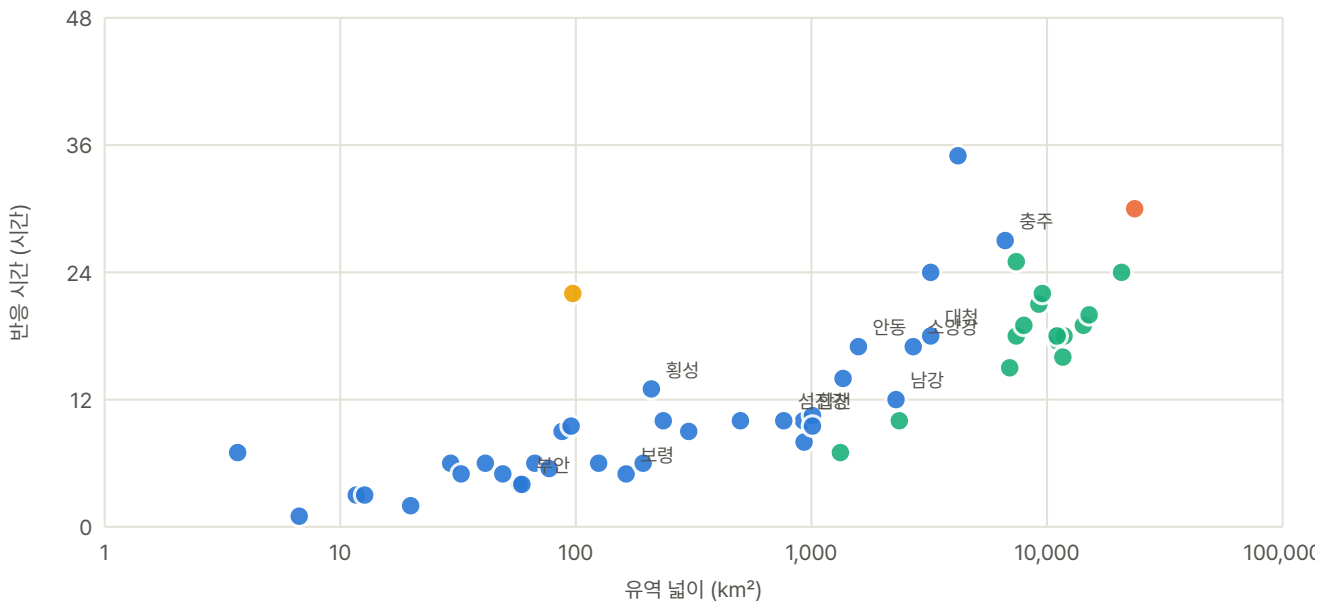
유입량이 컸던 사건 상위 8개 (전체 댐)

댐	비 최고	유입 최고	반응	사건 강우	최대 유입
창녕함안보	2020-08-08 02:00	2020-08-09 05:00	27시간	171mm	15,715m³/s
남강	2025-07-19 11:00	2025-07-19 15:00	4시간	310mm	15,292m³/s
남강	2025-07-17 15:00	2025-07-19 15:00	48시간	219mm	15,292m³/s
낙동강하굿둑	2020-08-08 02:00	2020-08-09 11:00	33시간	166mm	14,080m³/s
군남	2020-08-02 23:00	2020-08-05 22:00	71시간	256mm	13,681m³/s
군남	2020-08-04 11:00	2020-08-05 22:00	35시간	55mm	13,681m³/s
군남	2020-08-05 09:00	2020-08-05 22:00	13시간	121mm	13,681m³/s
합천창녕보	2020-08-07 15:00	2020-08-09 02:00	35시간	142mm	13,367m³/s

분석 ② 왜 댐마다 다를까 — 유역 넓이와 비교

점 하나 = 댐 하나. 가로 = 유역 넓이(비가 내려 이 댐으로 흘러드는 땅의 넓이, 눈금 한 칸마다 10배), 세로 = 반응 시간. 점이 오른쪽 위로 갈수록 “넓을수록 늦다”는 뜻입니다.

■ 댐 ■ 보 ■ 조정지 ■ 하굿둑



댐만 (35곳)

$\rho = 0.89$

순위 상관 · 우연일 확률
0.1% 미만

보·조정지 포함 (53곳)

$\rho = 0.86$

순위 상관 · 우연일 확률
0.1% 미만

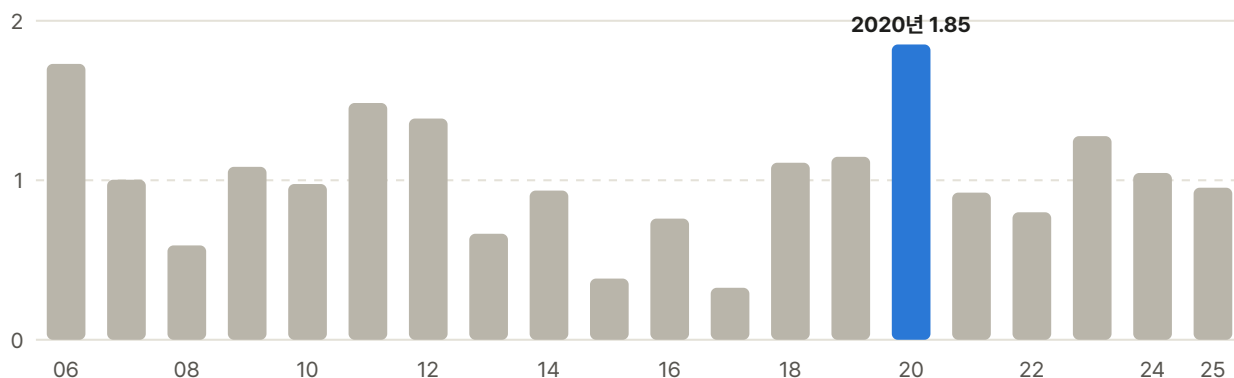
한 줄 해석

넓은 깔때기일수록 물이 가운데로 다 모이는 데 오래 걸리듯, 유역이 넓을수록 늦게 물려옵니다. ρ 는 “넓이 순”과 “반응 시간 순”으로 줄 세웠을 때 순서가 얼마나 같은지로, 1이면 똑같고 0.89는 거의 같은 순서입니다.

⚠ 한계: 함께 움직인다고 원인이라는 증명은 아닙니다. 경사, 상류 댐의 방류 같은 다른 이유도 섞여 있습니다. 발표에선 “관계가 뚜렷이 보인다”로 표현하세요. 보·조정지는 상류 댐 방류를 받는 곳이라 따로 보는 것이 맞습니다.

분석 ③ 20년 동안 홍수가 커졌을까

막대 하나 = 한 해의 홍수 크기: 그해 가장 많이 들어온 양 ÷ 그 댐의 보통 해 값(1 = 보통 해, 2 = 보통의 2배). 20년치 일 자료가 다 있는 댐 38곳을 모아 가운데 값으로 그렸습니다.



홍수 크기의 흐름

$\tau = -0.04$

+면 커짐, -면 작아짐,
0이면 흐름 없음 · $p =$
0.82: 우연으로도 흔히
나옴

한 해 강우량의 흐름

$\tau = 0.07$

$p = 0.68$

댐별로 따로 보면

36/38곳

뚜렷한 추세 없음 (감소
2곳)

결과를 이렇게 쓰면 됩니다

20년은 홍수가 커졌는지 말하기엔 짧고, 홍수 크기는 해마다 크게 출렁입니다(2020년 1.85, 2017년 0.33). “늘었다”를 주장하는 대신 **특별했던 해(2020·2006·2023)의 반응 시간·예측 성능이 평소와 어떻게 달랐나**를 비교하는 쪽이 과제의 “추세·비교” 요구를 더 잘 채웁니다. 이것도 “한계와 예외를 고려했다”는 증거가 됩니다.

분석 ④ 몇 시간 뒤 유입량을 맞힐 수 있나

2020-2022년으로 배우고 2023년으로 기준을 정하고 2024-2025년으로 시험했습니다. 숫자 = “지금 양 그대로” 찍기보다 덜 틀린 정도. 예: +10% = 찍기가 100만큼 틀릴 때 모델은 90만큼 틀림. 0보다 작으면 찍기보다 못함.

	1차 시도 (매시간 값)	2차 (3시간 평균)	2차 · 홍수 때만 (그 댐 유입 상위 5% 시간)	찍기를 이긴 댐 (2차)
3시간 뒤	+5%	+13%	+22%	38/49곳
6시간 뒤	+7%	+10%	+17%	38/50곳
12시간 뒤	-1%	+6%	+15%	28/51곳

6시간 뒤 예측, 댐마다 (점 하나 = 댐 하나, 오른쪽일수록 덜 틀림, 빨강 = 찍기보다 못함)



디버깅 기록: 1차 → 2차

1차는 매시간 유입량을 그대로 맞혔더니 찍기와 거의 같았습니다 (6시간 +7%). 원인: 유입량이 계산값이라 1시간 단위로 들쭉날쭉해 **맞힐 수 없는 흔들림**이 오차 대부분. → 지난 3시간 평균을 맞히게 바꾸고, 찍기도 똑같이 3시간 평균으로 맞춤.

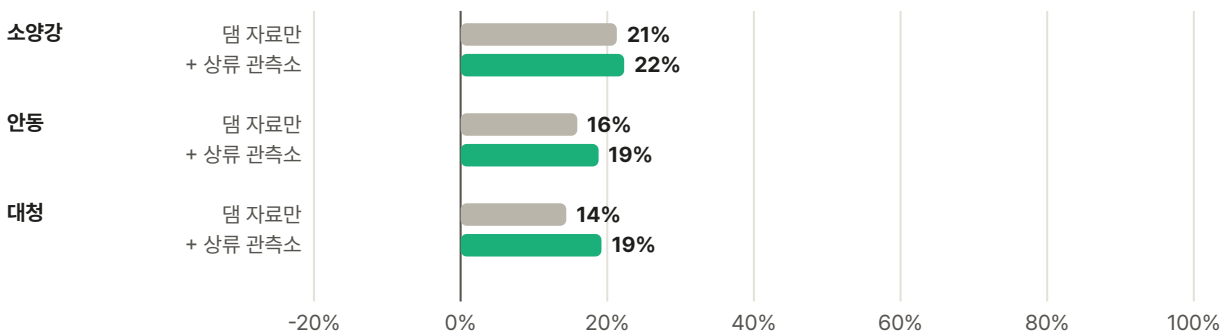
잘 맞은 곳 / 안 맞은 곳

잘 맞음: 충주조정지(48%), 안동조정지(47%), 합천조정지(38%), 대청조정지(33%)
안 맞음: 영천(-48%), 광동(-69%), 합천(-80%)
조정지는 바로 위 댐의 방류를 받아 예측이 쉽고, 0 이하가 많은 댐은 어렵습니다. 유입이 거의 없는 댐은 채점에서 뺐습니다.

분석 ⑤ 상류 비·강 관측소를 넣으면 더 잘 맞을까

WaterPulse의 간판인 “비 → 강 → 댐” 흐름을 시험했습니다. 소양강·대청·안동댐에 상류 관측소 자료를 더했습니다.

6시간 뒤 예측이 짝기보다 덜 틀린 정도 (길수록 좋음)



댐	3시간 뒤	6시간 뒤	12시간 뒤
소양강	+16% → +18%	+21% → +22%	+20% → +22%
안동	+22% → +23%	+16% → +19%	+10% → +11%
대청	+13% → +16%	+14% → +19%	+17% → +22%

넣은 상류 정보

- 상류 비 관측소 평균 강우 (1·3·6·12·24시간 합)
- 상류 강 수위의 1·3시간 변화(관측소 평균)
- 상류 강 유량 합과 3시간 변화

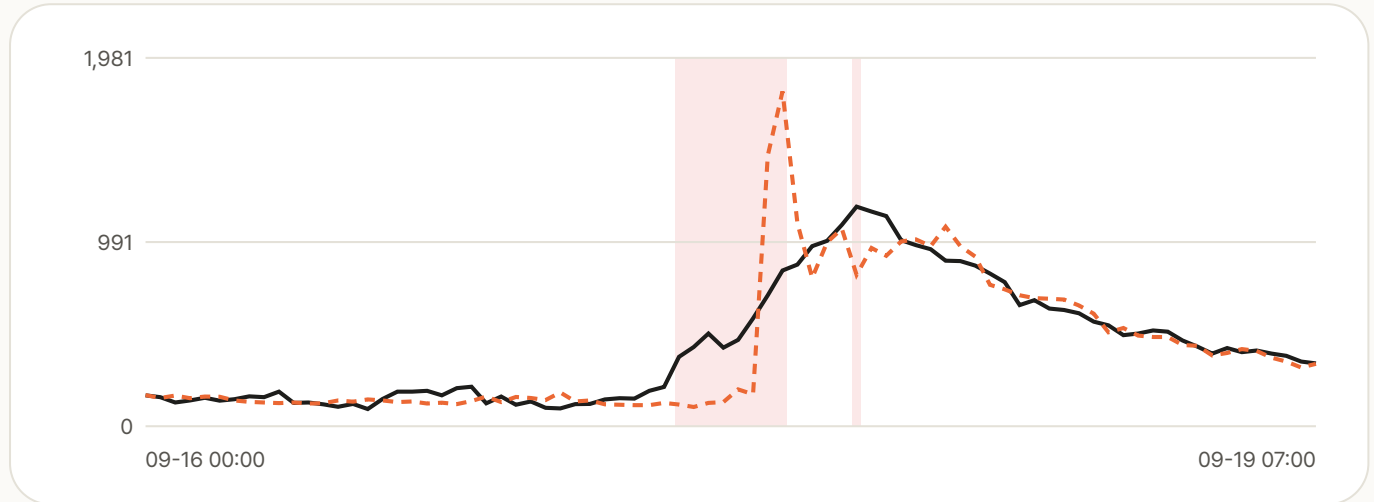
한 줄 해석

상류 정보를 넣으면 3개 댐 모두에서 예측이 나아졌습니다. 간판 아이디어가 실제 데이터로 뒷받침됩니다.

분석 ⑥ 평소와 다른 순간 찾기

6시간 전에 한 예측과 실제의 차이가 평소보다 훨씬 클 때(2023년에 나온 차이 중 가장 큰 0.5%보다 더 클 때) “평소와 다름”으로 표시했습니다.

● 실제(3시간 평균) ● 6시간 전에 한 예측 ■ 평소와 다름



2024-2025년 표시
횟수

1,157번

댐 59곳 합계

소양강댐 예시

2025-09-17

06:00

가장 크게 빗나간 차이
-964m³/s

주의

표시 = 사고가 아니라
예측이 크게 빗나간
순간입니다. 계산 오류
(0 이하 급변)인지,
갑작스런 방류·
집중호우인지 원인을
나눠 적어야 합니다.

실제 과제에서는 이렇게 옮기면 됩니다

노트북에서 한 것과 실습 VM에서 할 것을 짝지었습니다. 순서와 코드의 뼈대는 그대로 씁니다.

이번 시뮬레이션	실제 과제 (실습 VM)	수업
수집 폴더 (sim/spool)	→ VM의 로컬 수집 폴더	같음
flume_sim.py	→ Flume spooldir → HDFS sink	9장
lake/etl/.../raw	→ HDFS /etl/waterpulse/raw	2~3장
Spark SQL 외부 표 (USING json)	→ Hive CREATE EXTERNAL TABLE	5~6장
Spark SQL 조회	→ Impala 조회 (+ REFRESH)	5~6장
Parquet + partitionBy(year, month)	→ Hive/Impala STORED AS PARQUET, PARTITIONED BY	7~8장
Spark 로컬 모드	→ Spark on YARN (실습 VM)	10~17장
out/*.json	→ Sqoop export → MySQL	4장
GitHub Pages	→ 웹 서버 (MySQL 읽기)	-

✅ 이번 배운 것 → 계획에 반영할 것

1. 관측소 API는 **500줄 · 3갈래 · 0.3초 간격**으로
2. 시간 유입량은 **3시간 평균**으로 맞추기
3. **0 이하 유입량** 처리 규칙을 품질표와 함께 공개
4. 추세는 “늘었다” 대신 **특별했던 해 비교**로

5. 원본 표는 전부 STRING, 숫자 변환은 Spark에서
6. **불가능한 값**(평소 최대의 10배 + 한 시간만 틈)은 정리 단계에서 빈칸으로
7. 실습 VM의 Spark 버전·엄격 모드 확인
8. 상류 관측소는 3개 댐에서 효과 확인 → 대상 댐 늘리기
9. 칸 나누기·Parquet 효과는 VM에서 다시 재서 발표

🔄 똑같이 다시 돌리는 법

```
cd 과제/WaterPulse/sim
python3 collect.py all          # ① 수집 (키: 과제/.env, 약 38분)
python3 flume_sim.py           # ② 적재
JAVA_HOME=.../openjdk@21 .venv/bin/python etl_spark.py    # ③④ 정리·저장
JAVA_HOME=.../openjdk@21 .venv/bin/python analyze.py      # ⑤ 분석 (약 10분)
python3 collect_stats.py && python3 build_web_data.py      # 웹용 요약
python3 ../report_sim/build_sim_report.py                 # 이 문서
```

결과 웹페이지

<https://keemlogan.github.io/waterpulse/> · 토스 앱처럼 “한 화면에 한 가지”만 보이게 만들었습니다.

분석 결과 방류 계산기 재난 상황판

WaterPulse · 실데이터 사전 시뮬레이션

비가 오면, 댐에는 보통 10시간 뒤에 물이 가장 많이 몰려와요

K-water 댐·보 59곳의 2020~2025 시간 자료
3,103,843줄에서 큰비 사건 3,344개를 찾아 확인했어요.
2026년 10월 5일에 공공 API로 직접 받은 실제 값이에요.

694만 줄

공공 API 16,146번
호출로 받은 실제 값

38/50곳

6시간 뒤 예측이
“그대로 찍기”보다
정확한 댐

+3%p

상류 관측소를
넣으면 6시간
예측이 좋아지는 폭
(퍼센트포인트)

댐별 반응 시간

비가 가장 세게 내린 때부터 댐에 물이 가장 많이 들어올 때까지 걸린
시간이에요. 큰비마다 켜 시간을 줄 세운 한가운데 값이고, 열린 막대는
큰비 절반이 들어가는 범위예요.

Q 댐 이름 검색

빠른 순

느린 순

이름 순

전체

댐

보

하국둑

조정지

안계

댐 · 유역 7km² · 큰비 20번

1시간

1~16시간 >

분석 결과 방류 계산기 재난 상황판

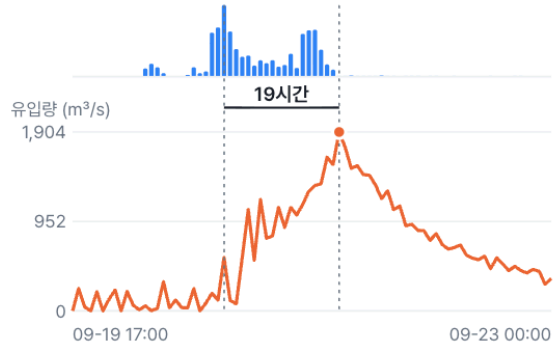
소양강 · 댐

큰비가 오면 약 17시간 뒤에 물이 가장 많이 들어와요

큰비가 왔을 때

2024-09-20 큰비 · 최대 1,904m³/s · 이 댐의 보통 반응 시간에 가장
가까운 큰 사건

비 (mm/시간)

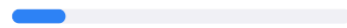


비가 가장 췄던 09-20 18:00 → 유입이 가장 많았던 09-21 13:00,
19시간 차이

몇 시간 뒤를 맞힐 수 있을까요

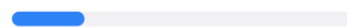
2024~2025년으로 시험했어요. “지금 양이 그대로 간다”고 찍을
때보다 덜 틀린 정도예요. 20%면 찍기가 100만큼 틀릴 때 모델은
80만큼만 틀렸다는 뜻이에요.

3시간 뒤



16%

6시간 뒤



21%

첫 화면

결론 한 문장과 숫자 세
개. 댐 목록은 반응
시간이 빠른 순.

댐을 누르면

가장 큰 비가 왔을 때의
그래프, 3·6·12시간
예측 성능, 데이터 상태.

공개한 것

결과 요약 JSON과
화면 코드뿐. 원본
데이터·인증키·작업
기록은 올리지
않았습니다.